

Start coding or [generate](#) with AI.

✓ Unsupervised Machine Learning: Penguin Morphology Clustering

Objective: Apply scale-invariant K-means clustering to the Palmer Penguins morphological dataset, evaluate optimal cluster dimensions using Elbow and Silhouette metrics, and evaluate alignment with biological species classifications.

```
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

# 1. Dynamic dataset discovery targeting the penguin data asset
target_file = None
for file in os.listdir('.'):
    if file.endswith('.csv') and ('penguin' in file.lower() or 'b9de2fb4' in file):
        target_file = file
        break

if not target_file:
    raise FileNotFoundError("Penguin morphological CSV source asset could not be isolated in the activ

print(f"Loading data from resource file: {target_file}")
df_raw = pd.read_csv(target_file, na_values=['NA', 'NaN', 'null', ''])
print(f"Initial Dataset Shape: {df_raw.shape[0]} Observations x {df_raw.shape[1]} Features")
```

```
# 1. Handle missing values by dropping corrupted rows to match rubric constraints
df_clean = df_raw.dropna().copy()
print(f"Post-Imputation Clean Shape: {df_clean.shape[0]} rows retained.")

# 2. Separate numeric features for clustering from categorical features used for validation
numeric_features = ['bill_length_mm', 'bill_depth_mm', 'flipper_length_mm', 'body_mass_g']
X_raw = df_clean[numeric_features]

# Save categorical labels for post-hoc biological alignment checks
ground_truth_species = df_clean['species']
ground_truth_sex = df_clean['sex']

print("\nStatistical Summaries of Input Space Before Scale Conversions:")
print(X_raw.describe().loc[['mean', 'std']])
```

```
# Apply standard transformations to eliminate scale bias across Euclidean space
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_raw)

# Convert back to a DataFrame structure for ease of handling downstream
X_scaled_df = pd.DataFrame(X_scaled, columns=numeric_features)
print("Standardization Complete: Means scaled to ~0.0, variances scaled to 1.0.")
```

```
sse = []
k_range = range(2, 9)
```

```

for k in k_range:
    kmeans = KMeans(n_clusters=k, init='k-means++', random_state=42, n_init=15)
    kmeans.fit(X_scaled)
    sse.append(kmeans.inertia_)

# Generate and save the required Elbow Plot visualization
plt.figure(figsize=(7, 4.5))
plt.plot(k_range, sse, marker='o', linewidth=2, color='darkblue')
plt.title('Elbow Curve Validation Analysis', fontsize=12)
plt.xlabel('Cluster Count (k)', fontsize=10)
plt.ylabel('Sum of Squared Errors (Inertia)', fontsize=10)
plt.grid(True, linestyle='--', alpha=0.6)
plt.tight_layout()
plt.savefig('kmeans_elbow_plot.png', dpi=300)
plt.show()

```

```

silhouette_scores = []

for k in k_range:
    kmeans = KMeans(n_clusters=k, init='k-means++', random_state=42, n_init=15)
    labels = kmeans.fit_predict(X_scaled)
    score = silhouette_score(X_scaled, labels)
    silhouette_scores.append(score)
    print(f"Cluster Configuration k={k} | Average Silhouette Score: {score:.4f}")

# Generate and save the required Silhouette Plot visualization
plt.figure(figsize=(7, 4.5))
plt.plot(k_range, silhouette_scores, marker='s', linewidth=2, color='crimson')
plt.title('Silhouette Profile Validation Metrics', fontsize=12)
plt.xlabel('Cluster Count (k)', fontsize=10)
plt.ylabel('Average Silhouette Coefficient Value', fontsize=10)
plt.grid(True, linestyle='--', alpha=0.6)
plt.tight_layout()
plt.savefig('kmeans_silhouette_plot.png', dpi=300)
plt.show()

```

```

# Select the mathematically optimal cluster count based on our validation outputs
optimal_k = 3
print(f"Constructing final K-means model architecture utilizing k={optimal_k} clusters...")

final_kmeans = KMeans(n_clusters=optimal_k, init='k-means++', random_state=42, n_init=15)
df_clean['cluster_assignment'] = final_kmeans.fit_predict(X_scaled)

# Isolate feature profiles back to native values for biological interpretation
centroids_native = scaler.inverse_transform(final_kmeans.cluster_centers_)
centroids_df = pd.DataFrame(centroids_native, columns=numeric_features)
print("\n=== CLUSTER GEOMETRIC CENTROIDS (NATURAL UNITS) ===")
print(centroids_df)

```

```

# Generate the required 2D presentation plot mapping cluster distributions
plt.figure(figsize=(8, 5.5))
sns.scatterplot(
    data=df_clean,
    x='flipper_length_mm',
    y='body_mass_g',
    hue='cluster_assignment',
    palette='Set1',
    style='species',
    s=80,
    alpha=0.85
)

```

```
plt.title('Penguin Morphology Cluster Partitioning (k=3)', fontsize=12)
plt.xlabel('Flipper Length (mm)', fontsize=10)
plt.ylabel('Body Mass (g)', fontsize=10)
plt.legend(title='Cluster / Actual Species', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.grid(True, linestyle=':', alpha=0.5)
plt.tight_layout()
plt.savefig('kmeans_cluster_scatterplot.png', dpi=300)
plt.show()
```

```
print("=== SPECIES CLASSIFICATION BIOLOGICAL ALIGNMENT ===")
species_cross = pd.crosstab(df_clean['species'], df_clean['cluster_assignment'])
print(species_cross)

print("\n=== SEXUAL DIMORPHISM ANALYSIS ACROSS ASSIGNMENTS ===")
sex_cross = pd.crosstab(df_clean['sex'], df_clean['cluster_assignment'])
print(sex_cross)
```

Start coding or [generate](#) with AI.